

Sección 4 — Juegos

Métricas: el grafo de Hamming 1 del corpus, la tasa de éxito del backtracking, la entropía del Wordle y el determinismo por semilla, medidos sobre los motores reales

1. Qué se mide y cómo

Todas las cifras de este documento salen de **ejecutar los motores de producción** (src/engine/escalera.js y crucigrama.js) con `npm run simular-juegos` (simulacion/juegos-stats.mjs → docs/datos-juegos.json, mismo patrón que la comparación SM-2/Leitner de la Sección 2). No hay reimplementación paralela: si el motor cambia, las métricas cambian con él. Para la **escalera** se caracteriza el grafo de palabras del corpus (¿existen retos? ¿de qué dificultades?); para el **crucigrama** y la **sopa**, un barrido de 200 semillas por tamaño mide tasa de éxito, densidad y coste; para el **Wordle**, un solver voraz por entropía juega contra TODOS los secretos posibles de cada longitud.

2. Escalera: el grafo de Hamming 1 del corpus

2.1 Formalización

$G_L = (V, E)$ con V = palabras de longitud L del léxico (solo ASCII) y $\{u, v\} \in E \Leftrightarrow d_H(u, v) = 1$. Un reto de k pasos es un par con $\text{dist}_G(\text{origen}, \text{destino}) = k$; BFS certifica la distancia en $O(V + E)$. La construcción del grafo usa cubetas comodín (L máscaras por palabra): $O(n \cdot L)$ contra el $O(n^2 \cdot L)$ de comparar parejas.

2.2 El grafo medido, por longitud

L	Nodos	Aristas	Componentes	Gigante	Aisladas	Diámetro*	Dist. media*
3	97	136	11	62	5	19	6.08
4	268	277	95	115	75	16	6.88
5	445	315	223	64	176	12	4.46

* Diámetro y distancia media sobre pares alcanzables (dentro de cada componente). El grafo NO es conexo — el léxico de un corpus literario no es un diccionario exhaustivo — pero la **componente gigante** basta: el generador de retos solo elige pares a distancia exacta k , que por definición viven en la misma componente. Nótese la forma de campana con la longitud: con $L=3$ hay pocas palabras pero muy conectadas; con $L=5$ hay más palabras pero el espacio 26^5 está mucho más vacío y las vecinas escasean.

2.3 ¿Existen retos para cada dificultad?

L	Pares alcanzables	a distancia 3	a distancia 4	a distancia 5	a distancia 6
3	2033	237	234	194	180
4	6845	533	624	656	672
5	2832	413	415	397	329

Los cuatro niveles de dificultad que ofrece la UI (3–6 pasos) tienen cientos o miles de pares candidatos en cada longitud: el selector nunca cae en vacío. La cola de la distribución (hasta el diámetro, 12–19 según L) queda como margen para dificultades futuras.

3. Crucigrama: el backtracking medido

3.1 Formalización

Colocar n palabras entrelazadas es un problema de satisfacción de restricciones (variables = palabras; dominios = colocaciones legales; restricciones = coincidencia de letra en cruces, extremos libres, sin contactos laterales, ≥ 1 cruce por palabra). La búsqueda es en profundidad con retroceso: el anclaje en cruces reduce el dominio de cada variable de $O(\text{filas} \cdot \text{columnas} \cdot 2)$ a las casillas que ya contienen una letra de la palabra, y el orden «más cruces primero» prioriza tableros densos. Presupuesto de 5.000 nodos y degradación $n \rightarrow n-1$ garantizan terminación con salida.

3.2 Barrido de semillas

n pedido	Semillas	Tasa de éxito	Palabras medias	Cruces medios	Densidad media	ms / tablero
6	200	100.0%	6	5.08	0.285	0.47
8	200	100.0%	8	7.15	0.268	0.44
10	200	100.0%	10	9.19	0.255	0.6

Con el pool real (400 entradas, 30 candidatas barajadas por semilla) el backtracking coloca las n palabras pedidas en el **100% de las semillas** probadas y en décimas de milisegundo: el presupuesto de nodos es un seguro, no un coste. Cruces medios $\approx n - 1$ con excedente pequeño: el tablero típico es un árbol de palabras con algún cruce extra — exactamente lo que las reglas permiten sin forzar rejillas imposibles. La densidad (casillas ocupadas / rectángulo envolvente) baja suavemente con n : tableros más grandes se dispersan, pero no colapsan.

4. Adivina la palabra: la entropía medida

4.1 Formalización

Sea S el conjunto de candidatas. Un intento g induce la partición de S por patrones de feedback (unión disjunta de clases $S_p = \{s : \text{feedback}(s, g) = p\}$). La información esperada de g es la entropía de Shannon $H(g) = -\sum (|S_p|/|S|) \cdot \log_2(|S_p|/|S|)$; el máximo teórico con $|S|$ candidatas es $\log_2|S|$ (partición en clases unitarias: el intento identificaría el secreto de golpe). El solver voraz juega el argmax de H sobre las candidatas consistentes en cada turno — voraz, no óptimo: el árbol de decisión óptimo exige minimizar la profundidad esperada, no la ganancia del turno (véase §10).

4.2 Resultados por longitud

L	Palabras	Mejor 1er intento	H (bits)	máx. $\log_2 S $	Intentos medios	Resuelto en ≤ 6
4	268	«nahe»	4.166	8.066	3.369	99.2%
5	445	«heran»	5.608	8.798	3.029	99.8%

Distribución de intentos del solver ($L=4$): 1: 1, 2: 37, 3: 122, 4: 87, 5: 14, 6: 5, 7: 2. ($L=5$): 1: 1, 2: 89, 3: 266, 4: 77, 5: 10, 6: 1, 7: 1.

Dos lecturas: (1) el mejor primer intento aporta $\sim 4.2/5.6$ bits de los $\sim 8.1/8.8$ posibles — un solo feedback no identifica el secreto, pero recorta el espacio en un factor $2^H \approx 18\text{--}49$; (2) el solver voraz resuelve casi todo el diccionario dentro de los 6 intentos del juego (99.3–99.8%) con ~ 3 de media: la cota de 6 intentos de la UI es holgada pero no trivial, señal de dificultad bien calibrada. El contador «quedan N posibles» de la UI muestra al alumno exactamente $|S|$ tras cada intento.

5. Sopa de letras: colocación aleatorizada medida

n pedido	Semillas	Sopas completas	Palabras medias
6	200	100.0%	6

n pedido	Semillas	Sopas completas	Palabras medias
8	200	100.0%	8
10	200	100.0%	10

La colocación aleatorizada (sin backtracking: sortear y reintentar) coloca las n palabras pedidas en el 100% de las semillas para n = 6–10 en la cuadrícula 12×12: con esa holgura de espacio, el retroceso sería maquinaria de más. El contraste con el crucigrama es el punto pedagógico: misma familia de problema, pero la restricción de conectividad (cada palabra debe cruzar otra) es la que obliga a retroceder.

6. Sudoku de palabras: unicidad y dificultad medidas

El sudoku usa como símbolos las 9 letras (todas distintas) de una palabra del corpus; una fila de la solución la lee completa. La generación es backtracking barajado + **excavado con unicidad**: cada vaciado se consiente solo si un contador de soluciones (MRV, corte en 2) sigue dando exactamente una. La **dificultad es del puzle** — las casillas dadas que quedan — no del vocabulario. El barrido genera 40 tableros por dificultad desde el pool real (40 palabras de 9 letras distintas) y re-verifica la unicidad de cada tablero de forma independiente:

Dificultad	Dadas objetivo	Dadas medias	Unicidad verificada	ms / tablero
Fácil	40	40	100.0%	4.6
Intermedio	32	32	100.0%	9.5
Difícil	26	26.1	100.0%	65.8

Fácil e intermedio alcanzan el objetivo exacto; en difícil el excavado se queda a veces una casilla corto (26,1 de media): no existía más vaciado que conservara la unicidad — el algoritmo prefiere un puzle honesto a uno con el número redondo. El coste crece con la dificultad (más llamadas al contador sobre tableros más vacíos) pero se mantiene en decenas de milisegundos: generable en el navegador al vuelo.

7. Codenames: la semilla como canal

La partida entera se codifica en 6 caracteres: 2 de vocabulario (idioma × nivel) y 4 de semilla en alfabeto de 36 símbolos → $36^4 = 1.679.616$ tableros por vocabulario. El LCG (a = 1103515245, c = 12345, m = 2^{31} , los parámetros de la libc clásica) genera la permutación de palabras (Fisher–Yates) y el reparto de colores 9/8/7/1. La propiedad que importa no es la calidad criptográfica (irrelevante aquí) sino el **determinismo multiplataforma**: la aritmética entera de JS reproduce el mismo tablero en cualquier dispositivo, y la semilla sustituye al servidor.

```
x_{i+1} = (a · x_i + c) mod m      con x_0 = hash(semilla)
36^4 = 1 679 616 tableros/vocabulario · 13 vocabularios
```

8. Los juegos por lectura: disponibilidad medida

Los cuatro juegos de palabras se pueden jugar con el vocabulario de UNA lectura: la navegación va por juego y, dentro de cada juego, se elige el vocabulario (el corpus entero o una lectura). La disponibilidad no es una lista curada: cada juego tiene un criterio formal en src/engine/juegos.js — escalera: existe un par a distancia ≥ 3 en el grafo de Hamming de la lectura; Wordle: alguna longitud con ≥ 12 palabras; crucigrama/sopa: ≥ 6 entradas con pista; sudoku: ≥ 1 palabra de 9 letras distintas. El índice de cada juego lista solo las lecturas que lo aguantan. La tabla sale de ejecutar esos criterios sobre los pools reales (pipeline/juegos.py agrupa las partes de un libro por título):

Lectura	Nivel	L=3/4/5	Entradas	Juegos disponibles
Anas Tag	principiante	9/5/5	17	crucigrama, sopa
Der Markt	principiante	8/6/4	10	crucigrama, sopa

Lectura	Nivel	L=3/4/5	Entradas	Juegos disponibles
Toms Wochenende	principiante	10/10/10	23	crucigrama, sopa
Der Hase und die Schildkröte	intermedio	13/16/11	32	crucigrama, wordle, sopa
Der Löwe und die Maus	intermedio	12/12/11	22	escalera, crucigrama, wordle, sopa
Der Nordwind und die Sonne	intermedio	16/11/12	38	crucigrama, wordle, sopa
Der süße Brei	intermedio	21/26/20	43	escalera, crucigrama, wordle, sopa
Die Ameise und die Grille	intermedio	15/15/8	45	escalera, crucigrama, wordle, sopa
Die Sterntaler	intermedio	23/16/20	44	crucigrama, wordle, sopa
Rotkäppchen	intermedio	35/44/45	100	crucigrama, wordle, sopa, sudoku
Die Verwandlung	avanzado	78/189/316	120	escalera, crucigrama, wordle, sopa, sudoku
Immensee	avanzado	86/186/274	120	escalera, crucigrama, wordle, sopa, sudoku

El patrón confirma la intuición pedagógica que motivó el diseño: las lecturas de principiante (30-40 palabras útiles) no sostienen ni el grafo de la escalera (sin pares a distancia 3) ni un diccionario de Wordle, pero sí crucigrama y sopa; las intermedias ganan el Wordle (y Rotkäppchen, el sudoku); los libros lo ofrecen todo. El criterio es una función del vocabulario, así que al añadir lecturas la tabla se actualiza sola.

9. Determinismo transversal

Los seis juegos comparten el mismo generador (board.js) y la misma disciplina que las Secciones 2 y 3: **el azar solo entra por la semilla**. Consecuencias medibles: los tests de los motores pueden afirmar igualdad exacta de retos y tableros (toEqual entre dos llamadas con la misma semilla); un reto de escalera o un crucigrama son compartibles por URL/string sin estado en servidor; y las estadísticas de este documento son reproducibles con `npm run simular-juegos`.

10. Limitaciones y trabajo futuro

El grafo hereda el corpus. Las glosas y las palabras vienen de lecturas literarias: hay formas conjugadas raras en la escalera y alguna glosa contextual («hoch» → «anticlón» si en el texto era el sustantivo Hoch). El filtro es léxico, no semántico. **Sin umlauts.** Excluir ä/ö/ü/ß simplifica el tecleo pero recorta el alemán real; una alternativa futura es aceptar ae/oe/ue como equivalentes. **Crucigrama sin rejilla simétrica.** Se genera estilo «entrelazado libre», no la rejilla simétrica de periódico (eso exigiría diccionarios mucho mayores y otro algoritmo, p. ej. dancing links). **Solver voraz, no óptimo.** Maximizar la entropía del turno no minimiza la profundidad esperada del árbol de decisión (el óptimo exacto es un problema de búsqueda en árboles con poda, resuelto por fuerza bruta en el Wordle inglés); para caracterizar la dificultad del diccionario, el voraz basta y es el estándar de referencia. **Futuro:** retos diarios (semilla = fecha), palabras de la bolsa del usuario como pool del crucigrama y de la sopa (el Codenames ya la acepta como vocabulario del tablero), y dificultad de la escalera por rareza de las palabras además de por distancia.